ESQUEMA GREEDY O VORAZ



Antonio Miguel Almagro Valles
Javier Díaz Vilchez
Rodrigo láñez Huertas

Objetivo **2**
Antecedentes **2**
Estudio de la implementación **3**
PrimSinPQ 4
PrimConPQ 5
Kruskal 5
TSPGreedy 5
TSPSimplePaths 5
Estudio teórico **5**
PrimSinPQ 5
PrimConPQ 6
Kruskal 6
TSPGreedy 6
TSPSimplePaths 7
Estudio experimental **7**
Anexo **9**
Arista 9
Network 10
MainEDALand 11
AlgoritmosEDALand 11
RedAleatoria 12
Utilities 12
AlgoritmoEDALandTest 13
Fuentes bibliográficas **14**

Objetivo

En esta práctica el método algorítmico utilizado es el llamado Greedy o Voraz. Un algoritmo Greedy, también conocido como algoritmo voraz, es una estrategia para resolver problemas que se basa en tomar la mejor decisión posible en cada etapa del proceso. En cada etapa, el algoritmo elige la opción que parece ser la más beneficiosa en ese momento específico, es decir, la mejor opción a corto plazo, con la expectativa de que estas decisiones individuales resulten en la mejor solución a largo plazo.

El objetivo de esta práctica es ofrecer una explicación detallada del funcionamiento de dicho método algorítmico. Basándonos en ejemplos bien ejecutados, explicando su resolución y haciendo un estudio exhaustivo del funcionamiento de cada uno de los algoritmos (Prim y Kruskal), además de la resolución de unos de los problemas más recurrentes en este tipo de casos (Problema del viajante de comercio).

Se proporcionarán los resultados obtenidos de dichos algoritmos, además de una comparativa de tiempos de ejecución, mediante la que se podrá hacer una estimación de cuál de los procesos es más eficiente. Javier Díaz Vílchez es nuestro jefe de equipo, el cual se encargará de supervisar y asignar las tareas a los miembros del equipo.

El desglose de las tareas a realizar ha sido el siguiente:

- Javier se ha encargado de realizar Prim, tanto con cola de prioridad como sin cola de prioridad y de la clase network y arista.
- Antonio ha realizado el algoritmo Kruskal y los tiempos de ejecución.
- Rodrigo hizo los algoritmos TSP Greedy y TSP Greedy Óptimo.
- Todos hemos ayudado en la parte de MainEDAland, red aleatoria y graphviz.

Hay que tener en cuenta que aunque cada uno se haya enfocado en una cosa, todo ha sido revisado por cada miembro del equipo para el correcto funcionamiento.

Antecedentes

Motivo de la realización de la práctica:

Se ha llevado a cabo la realización de esta práctica para determinar cuál es el mejor camino a seguir para instalar una red de fibra óptica por todo el país de EDALand. Este proceso se llevará a cabo por la empresa EDACom, que ha delegado el problema del camino a seguir a theBestSoft, que a su vez se lo ha encargado a EDASoft para que lo resuelva de manera posible.

Para ello, EDASoft ha decidido hacer una serie de simulaciones con un mapa más reducido de la región de EDALand para comprobar el correcto funcionamiento de sus algoritmos. Una vez se ha comprobado que todos funcionan bien, se ejecutarán los mismos procedimientos sobre la región completa de EDALand.

Otro problema al que debe de hacer frente la organización de EDACom, es ser capaz de llevar a cabo un proceso de mantenimiento sobre unos núcleos de población específicos. Para ello se le ha encargado de nuevo a EDASoft un nuevo problema, que determine el circuito con la distancia más corta que deba seguir dicha unidad de mantenimiento para satisfacer todos los tramos establecidos.

Estudio de la implementación

Para el estudio de la implementación en esta práctica partimos de un esquema básico inicial: tratamos con algoritmos voraces. Teóricamente, definimos los algoritmos voraces como métodos de resolución de problemas que conforman una estrategia en la cual en cada paso o etapa de estos, se elige la mejor opción local y que en conjunto de todas estas opciones nos lleva a una solución global óptima. Un algoritmo voraz selecciona la mejor opción en el momento sin considerar las consecuencias a largo plazo, lo que conlleva a que las soluciones que nos dan no logren finalmente el objetivo final de la manera más precisa, como veremos posteriormente para el algoritmo voraz desarrollado para el problema del viajante de comercio. Para esta práctica, vamos a trabajar con grafos no dirigidos, ponderados y conexos, de tal forma que la mejor opción siempre va a estar relacionada con la ponderación de las aristas, ya que el objetivo final de manera inicial será encontrar el árbol de expansión mínima (MST) dentro de una red de aristas ponderadas. Posteriormente, se nos propone otro objetivo final diferente en el cual deberemos afrontar como hemos dicho anteriormente el problema del viajante de comercio, para el cual utilizaremos la misma lógica que los anteriores algoritmos.

Como punto inicial hemos partido de una clase que administra nuestra red de aristas y vértices, de tal forma que podamos añadir vértices y aristas, comprobar si contiene un

vértice o arista en concreto, ver el número de vértices y aristas, comprobar el peso de una arista… Para esto, la mejor opción es usar la clase Network que implementamos anteriormente en la asignatura de Estructura de Datos y Algoritmos I. Esta clase tiene todo lo necesario para poder trabajar con una red de aristas ponderadas y vértices, además de que implementa un iterador sobre los vértices de la red. **El diseño de esta clase, al igual que todas las que se van a mencionar, se encuentra en el anexo de este documento y la documentación en JavaDoc en el código de github del repositorio del equipo.**

Como se puede observar en el diseño, la clase Network está parametrizada a un tipo genérico “Vertex” que representa los vértices de la red y que implementa la interfaz comparable. Para nuestro caso, nuestros vértices van a ser de tipo String ya que para el grafo de “graphEDAland”, trabajamos con cadenas de texto que representan ciudades de España. Esto nos viene bien ya que como sabemos “String” es una clase predefinida de java que implementa la interfaz comparable. Para el grafo de “graphEDAlandLarge”, aunque consideramos los vértices como formato numérico, seguiremos usando un tipo de dato “String” mediante la representación de los números en cadenas con el método string.valueOf().

Para definir las aristas, hemos realizado una clase llamada “Arista” con su constructor propio en función de los vértices de origen, destino y peso y una serie de métodos que nos permiten trabajar con ellas. Destacamos los getters y setter para los dos vértices y el peso, un toString() personalizado, el método equals que nos permite comparar la igualdad entre dos aristas en función de vértices de origen y destino iguales y el método compareTo() necesario para los algoritmos greedy, ya que compararemos las aristas en función de su peso.

Una vez visto cómo es la manipulación de las redes y cómo vamos a definir las aristas, pasamos a explicar los diferentes algoritmos para resolver los diferentes problemas propuestos en esta práctica.

Como hemos dicho anteriormente, tratamos con algoritmos voraces que van a encontrar en cada etapa la mejor solución posible. Esto nos indica hacer uso de algoritmos iterativos en los cuales en cada iteración se elige la mejor solución posible de entre las posibles soluciones. Los algoritmos los hemos desarrollado en su plenitud en una clase denominada “AlgoritmosEDAland”, y el estudio de su implementación viene a continuación:

PrimSinPQ

. Para el primer algoritmo, se propone realizar el algoritmo voraz de Prim para encontrar el árbol de expansión mínima sin utilizar una cola de prioridad. Teóricamente, el algoritmo de Prim para cada vértice elige al mejor vecino (aquel con el cual forme la arista de menor peso) y el resto de aristas las mantiene activadas. Esto debido a que el algoritmo intenta evitar que se formen ciclos a lo largo de su ejecución, de tal forma que al finalizar encuentre el árbol de expansión mínima y haya visitado todos los vértices del grafo.

Para ello, necesitamos de un vértice de origen del cual partiremos. Iteramos hasta que hayamos visitado todos los vértices y en cada iteración elegiremos la arista de menor peso,

partiendo desde este origen. Estas aristas se añadirán a una lista de aristas activadas, en la cual se van almacenando para comprobar, cómo hemos dicho anteriormente, que no se formen ciclos con las aristas solución. Dentro de esta lista de aristas activadas, en cada iteración se buscará la de menor peso y se añadirá al conjunto de aristas solución. Este conjunto será el que devolverá el método al finalizar, que será cuando una lista que hemos definido como 'verticesSol' tenga el mismo tamaño que los vértices de la red inicial, lo que indicaría que hemos recorrido todos los vértices.

PrimConPQ

. Para este segundo algoritmo, realizamos Prim pero implementado de una manera más eficiente con una cola de prioridad. Esta cola de prioridad la utilizaremos para las aristas activadas, de tal manera que las de menor peso serán las de mayor prioridad dentro de la cola. Al utilizar la función poll() de la cola de prioridad, sacará la arista de menor peso (o mayor prioridad) y no necesitamos iterar sobre la lista como hemos hecho anteriormente para el algoritmo sin cola de prioridad. De este modo mejora la eficiencia con respecto al anterior algoritmo al reducir un bucle.

Kruskal

. Teóricamente, el algoritmo de Kruskal funciona de modo distinto pero similar al algoritmo de Prim, aunque llega al mismo resultado de árbol de expansión mínima. Esta vez, el algoritmo itera sobre todas las aristas del grafo, de tal manera que en cada iteración se escoge siempre la menor de todo el grafo, buscando al igual que en el algoritmo de Prim que no se formen ciclos con las aristas solución y llegando al mismo árbol de expansión mínima. Como iteramos sobre todas las aristas, no tenemos vértice de origen.

Para la implementación, hemos definido al igual que en el caso anterior para Prim con una cola de prioridad, una cola de prioridad que contendrá todas las aristas del grafo. Para ello, iteramos inicialmente sobre el grafo completo y añadimos todas las aristas a la cola de prioridad. Una vez hemos añadido todas las aristas a la cola, iteramos al igual que en el caso de Prim hasta que hayamos recorrido todos los vértices de la siguiente forma:

Durante cada iteración, utilizamos una función recursiva llamada `buscarRaiz` para determinar si los vértices de una arista están en el mismo conjunto de aristas, lo que denominamos como una operación de unión de conjuntos disjuntos. Esto nos ayuda a evitar la formación de ciclos en el árbol de expansión mínima. Si los vértices no están en el mismo conjunto, significa que agregar esta arista no creará un ciclo, por lo que la añadimos al conjunto de aristas solución.

Este enfoque garantiza que seleccionemos las aristas más pequeñas disponibles en cada paso sin crear ciclos en el árbol resultante, lo que nos lleva a obtener un árbol de expansión mínima para el grafo dado.

TSPGreedy

. Para este algoritmo se plantea una solución voraz para el problema del viajante de comercio. Teóricamente este problema tiene un enfoque simple: visitar todos los vértices del grafo una sola vez partiendo desde un origen, es decir, buscar el camino hamiltoniano dentro de un grafo, por lo que al igual que en los anteriores algoritmos debe ser primeramente conexo. Sin embargo, esto no garantiza que el grafo tenga camino hamiltoniano, por lo que para este problema puede haber o no solución.

La implementación propuesta debe ser un algoritmo voraz para este caso. Por ello, hemos implementado un algoritmo iterativo a partir de un origen que valora sus vértices vecinos en función del peso de la arista formada. Para buscar el mejor vecino, hemos definido una función 'verticesMasCercano()' la cual encuentra para cada vértice la mejor opción, es decir, la arista formada de menor peso. En el momento en el que el vértice actual no tenga ningún vecino al que visitar (porque ya se haya visitado anteriormente), el algoritmo termina lanzando una excepción que nos indica que el grafo no tiene solución, es decir, que no presenta camino hamiltoniano.

Las estructuras utilizadas son un HashSet que almacena los vértices visitados y una lista solución con las aristas recorridas que conforman el camino hamiltoniano.

TSPSimplePaths

. Este algoritmo presenta una solución para el problema del viajante de comercio pero de una forma más exhaustiva. Para este caso, buscamos encontrar la mejor solución final, por lo que el enfoque es distinto, buscando la mejora final y no tanto la mejor solución local. Para ello, se realiza un método de "vuelta atrás" o "backtracking", en el cual se valoran todos los posibles caminos hamiltonianos del grafo y se elige aquel cuyo peso es menor. Este enfoque es más exacto pero su problema fundamental es el incremento exponencial de su orden de complejidad en comparación con el algoritmo voraz, producido por este método de backtracking.

En cuanto a la implementación de este algoritmo, hemos definido una función denominada "TSPSimplePathsAux" que busca el mejor camino de entre los posibles caminos hamiltonianos encontrados para el grafo desde un vértice de origen. Este calcula al igual, que hemos realizado, anteriormente para el algoritmo voraz, la mejor solución para cada vértice actual, es decir, la arista de menor peso. Una vez encontrado el camino hamiltoniano, se compara con el mejor encontrado hasta ese momento y si el peso total es menor, ese camino pasa a ser el elegido, hasta que se han recorrido todos los caminos desde el vértice de origen.

Las estructuras utilizadas para este algoritmo son dos listas: una para los vértices visitados y otra que guarda el mejor camino hamiltoniano de aristas.

Para finalizar este apartado, mencionamos el algoritmo implementado para la generación de redes aleatorias, en el cual partiendo de un número de vértices, empleamos la expresión (n * (n - 1) / 2), siendo n el número de vértices, para calcular el número de aristas necesario que garantice que el grafo generado sea conexo. Las relaciones entre los vértices se realizan de forma aleatoria, al igual que el peso generado con un valor de entre 1 y 10000. Para garantizar la conectividad del grafo, hemos elaborado también un método denominado "comprobarConectividadRed" que itera sobre los vértices del grafo y comprueba que todos tengan al menos un vértice vecino.

Estudio teórico

PrimSinPQ

Este algoritmo tiene una complejidad **O(n*m)** donde **n** es el número de vértices y **m** el de aristas. El algoritmo consiste en un bucle while exterior, que itera sobre el número de vértices hasta n-1. Dentro del bucle, hay cuatro bucles for independientes, el primero se encarga de iterar sobre todos sus vecinos, es decir, todas sus aristas, el siguiente se encarga de añadir las aristas al conjunto **aristasActivadas**. Luego se busca la arista de menor coste y se actualizan los conjuntos **verticesSol** y **aristasActivadas**.

Por tanto, la complejidad final del algoritmo es **O(n m)**, ya que cada vértice puede tener hasta m aristas y estas operaciones se realizan para n vértices

PrimConPQ

Este algoritmo se implementa de una manera muy parecida al anterior. Hay como en el que no usa colas de prioridad un bucle while externo que recorre los vértices. La diferencia se encuentra dentro, porque ya que como se usan colas de prioridad, el orden de complejidad de la operación encargada de encontrar el coste mínimo de entre todas las aristas no es **m**, sino **log m**.

Por tanto, la complejidad final del algoritmo es **O(n log m)**. Esta complejidad refleja el hecho de que para cada uno de los n vértices, se realiza una operación de extracción del mínimo con una complejidad de **O(log m)**. Esto hace que el algoritmo sea más eficiente en comparación con la versión que no utiliza una cola de prioridad.

Kruskal

Este algoritmo está compuesto por dos bucles for anidados, el primero se encarga de iterar sobre todos lo vértices y el segundo itera sobre el número de aristas de cada vértice y añade todas las aristas a la cola de prioridad. Esta parte del código, tiene un orden de complejidad **O(m log m)** ya que cada operación de inserción a una cola de prioridad tiene orden **O(log m)**. A continuación hay un bucle while que itera sobre todas las aristas y se realiza una operación poll para extraer la arista de menor peso, dicha operación tiene un orden de complejidad logarítmico, por lo que el orden del bucle while sería **O(n log m)**.

Por tanto, el orden de complejidad final del algoritmo sería **O(m log m + n log m)**. Sin embargo, en nuestro caso, el uso de la técnica de compresión de caminos en el método **buscarRaiz()**, puede reducir la complejidad de esta función a cerca de **O(1)**, lo que haría que la complejidad final del algoritmo sea **O(m log m)**.

TSPGreedy

El orden de complejidad de este algoritmo es **O(n^2)** ya que el método consta de un único bucle while de orden **O(n)**, que itera sobre todos los vértices. Sobre cada vértice, hay una llamada a la función **verticeMasCercano()**, que tiene el mismo orden de complejidad, **O(n)**.

Por tanto, el orden de complejidad final del algoritmo es **O(n^2)**.

TSPSimplePaths

Este algoritmo es de un orden de complejidad **O(n!)**, el algoritmo principalmente llama al método **TSPSimplePathsAux()**, el cual consta principalmente de un bucle for que se ejecuta hasta que se han visitado todos los vértices, dentro del bucle se vuelve a llamar a la función **TSPSimplePaths()**, que se ejecuta para cada vértice. La complejidad de este bucle es **O(n!)**, ya que hay n! permutaciones posibles de los vértices a visitar.

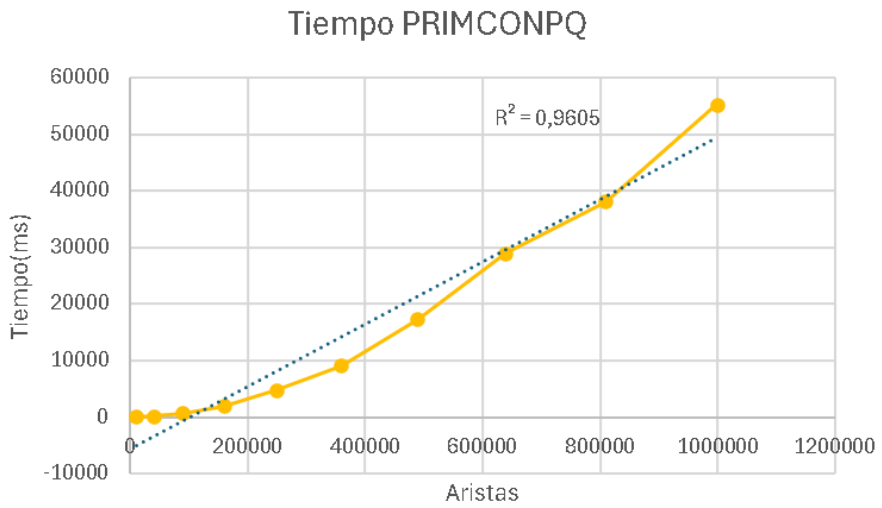
Por tanto, el orden de complejidad final del algoritmo es **O(n!)**, esto se debe a su naturaleza de encontrar todos los caminos posibles, y una vez encontrados, decidir cuál es el más óptimo. Lo bueno de este algoritmo es que te garantiza el mejor camino a tu solución, lo malo es que es inviable para grafos de tamaño medio/grande, pero ideal para grafos pequeños.

Estudio experimental

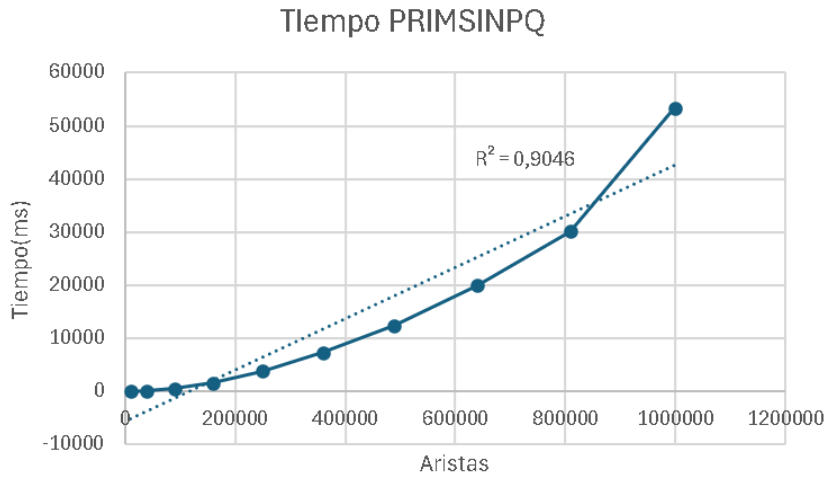
En este apartado vamos a desarrollar la validación de la implementación de los algoritmos, mediante pruebas con datos aleatorios en función del algoritmo que hemos realizado para generar grafos aleatorios para distintos valores de n, donde n son el número de vértices. Para ello realizaremos el análisis de los tiempos de ejecución de cada algoritmo y se representarán con sus correspondientes gráficas.

Para saber si es correcto el proceso de toma de muestras se usará el estadístico R^2, si está entre 0 y 1 significa que la muestra ha sido tomada correctamente.

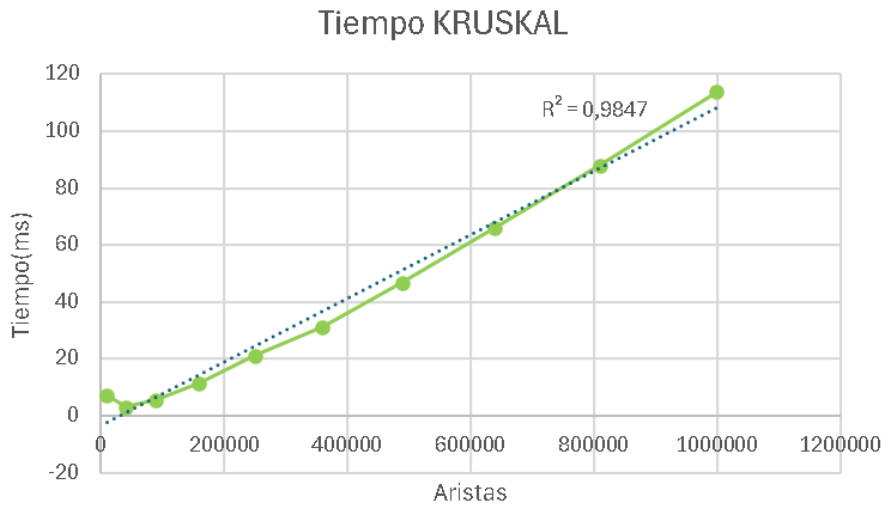
ALGORITMO PRIM CON PRIORITY QUEUE



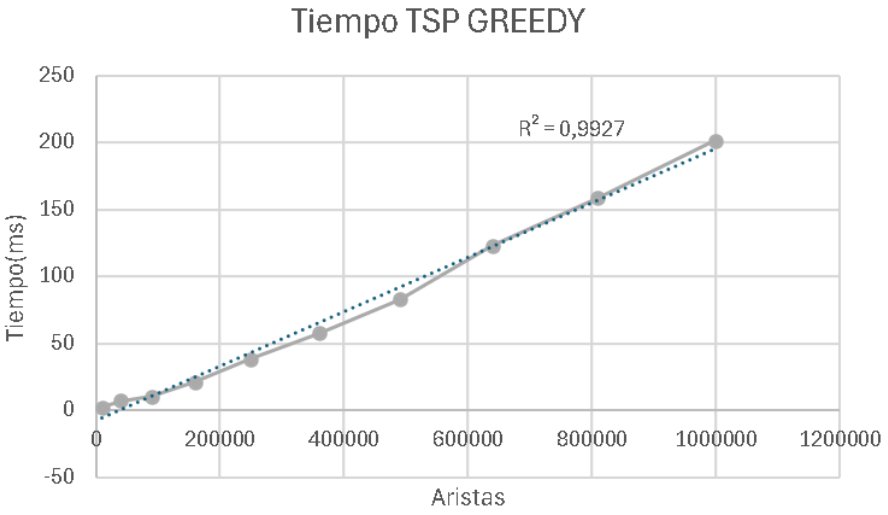
ALGORITMO PRIM SIN PRIORITY QUEUE



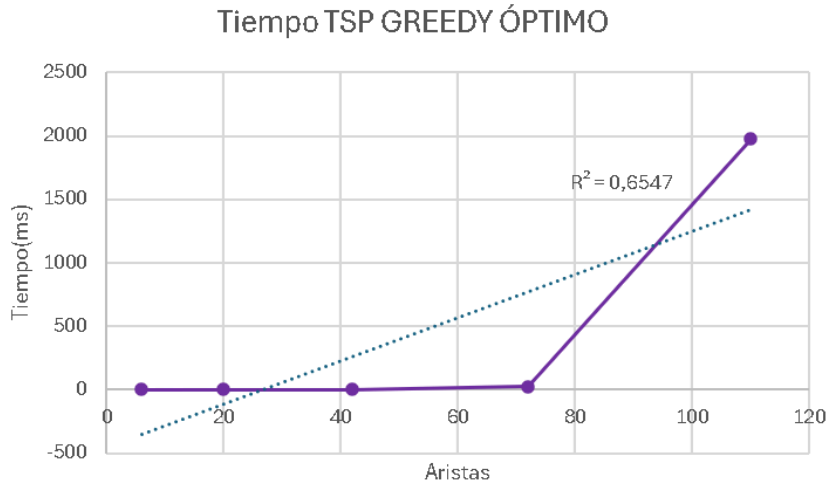
ALGORITMO KRUSKAL



ALGORITMO TSP GREEDY

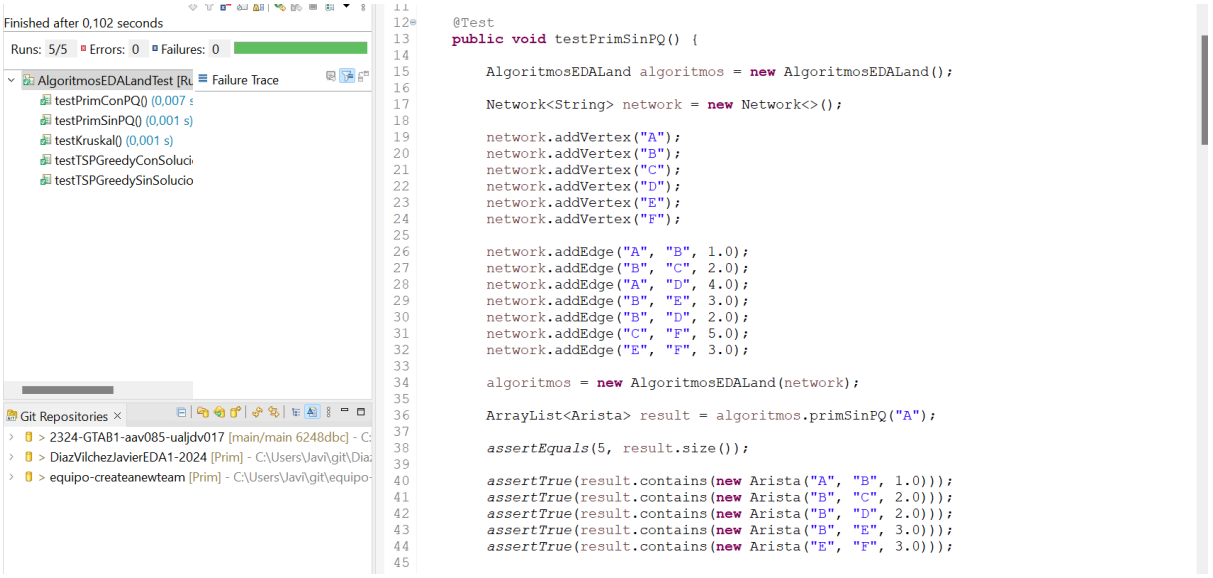


ALGORITMO TSP GREEDY ÓPTIMO



En este caso hemos tenido que realizar una pruebas de rendimiento a parte, ya que a a partir de los 11 vértices y 110 aristas, tardaba más de 30 minutos

Como se puede observar todos están por encima de 0,90, y algunos por encima de 0,95, menos el TSP de caminos simples, ya que su orden de complejidad es de n! como hemos explicado en el apartado teórico. En cuanto al funcionamiento correcto de estos, en base a los test que hemos realizado para distintos grafos que hemos formado, vemos la salida de los test en verde:




```java
@Test
public void testPrimSinPQ() {

    AlgoritmosEDALand algoritmos = new AlgoritmosEDALand();

    Network<String> network = new Network<>();

    network.addVertex("A");
    network.addVertex("B");
    network.addVertex("C");
    network.addVertex("D");
    network.addVertex("E");
    network.addVertex("F");

    network.addEdge("A", "B", 1.0);
    network.addEdge("B", "C", 2.0);
    network.addEdge("A", "D", 4.0);
    network.addEdge("B", "E", 3.0);
    network.addEdge("B", "D", 2.0);
    network.addEdge("C", "F", 5.0);
    network.addEdge("E", "F", 3.0);

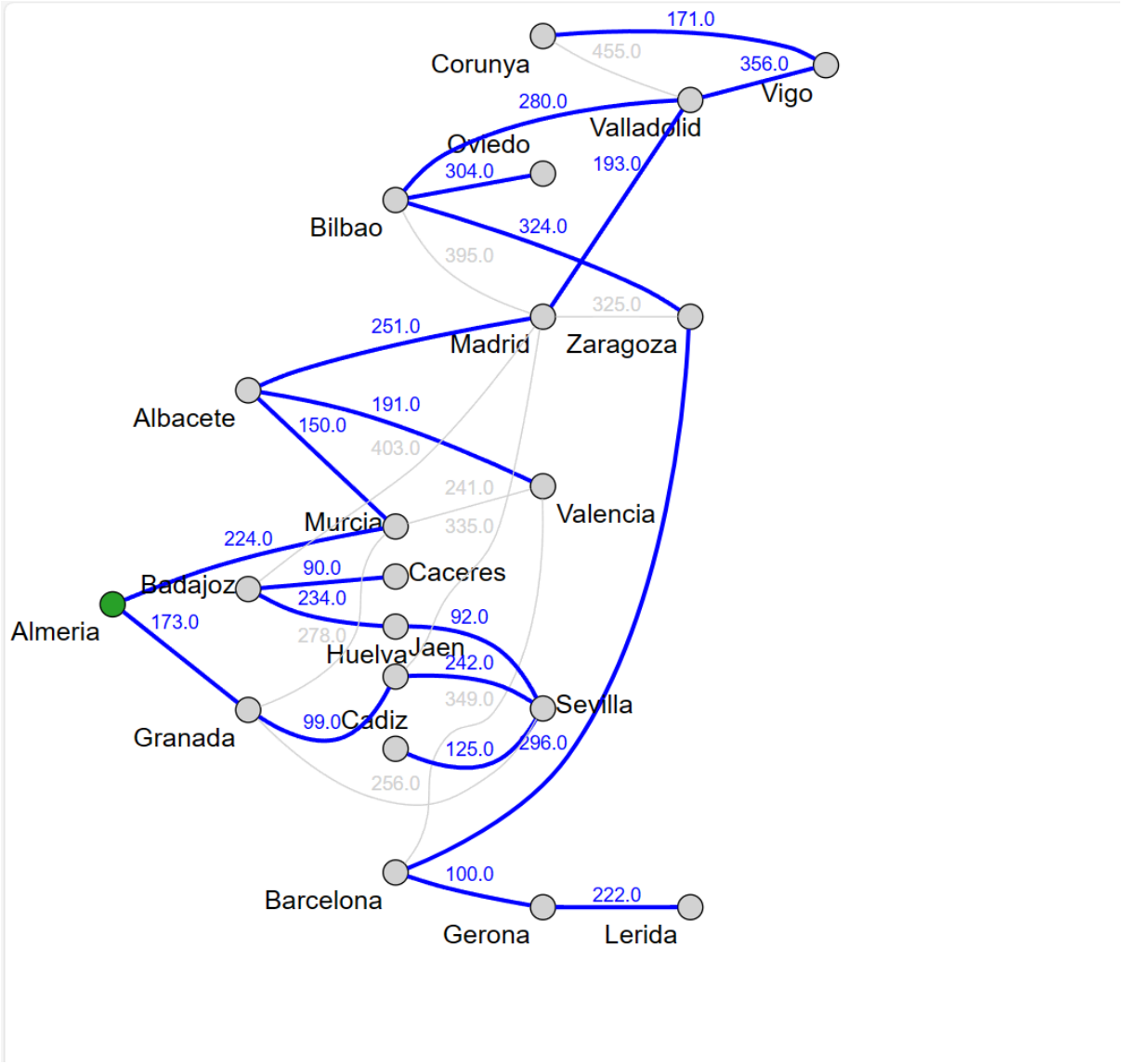
    algoritmos = new AlgoritmosEDALand(network);

    ArrayList<Arista> result = algoritmos.primSinPQ("A");

    assertEquals(5, result.size());

    assertTrue(result.contains(new Arista("A", "B", 1.0)));
    assertTrue(result.contains(new Arista("B", "C", 2.0)));
    assertTrue(result.contains(new Arista("B", "D", 2.0)));
    assertTrue(result.contains(new Arista("B", "E", 3.0)));
    assertTrue(result.contains(new Arista("E", "F", 3.0)));
```

Para los grafos generados en graphViz, vemos el ejemplo para el grafo de “graphEDAland” creado, en el que se puede observar el árbol de expansión mínima remarcado en color azul y el resto de aristas:



Anexo

. Para este anexo, hemos incluído el diseño de cada una de las clases implementadas para esta práctica donde desglosamos los métodos de cada una y una breve descripción:

- Clase **`Arista.java`**: Esta clase representa una arista (o borde) en un grafo. Una arista es una conexión entre dos vértices en un grafo. En la clase Arista, se almacenan los siguientes datos:
    - **`vOrigen`**: El vértice de origen de la arista.
    - **`vDestino`**: El vértice de destino de la arista.
    - **`peso`**: El peso (o coste) asociado a la arista.

- **`Arista(String origen, String destino, Double peso)`**: Constructor que inicializa una arista con un vértice de origen, un vértice de destino y un peso.
- **`Arista()`**: Constructor vacío.
- **`toString()`**: Devuelve una representación en cadena de la arista.
- **`getvOrigen(), setvOrigen(String vOrigen)`**: Obtiene y establece el vértice de origen de la arista.
- **`getvDestino(), setvDestino(String vDestino)`**: Obtiene y establece el vértice de destino de la arista.
- **`getPeso(), setPeso(Double peso)`**: Obtiene y establece el peso de la arista.
- **`equals(Arista o)`**: Comprueba si esta arista es igual a otra arista o.
- **`compareTo(Arista o)`**: Compara esta arista con otra arista o en función del peso y los vértices.

- Clase **`Network.java`**: Esta clase representa un grafo. Un grafo es una estructura de datos que consiste en vértices (o nodos) y aristas (o bordes). En la clase Network, se almacenan los siguientes datos:

  - **`directed`**: Un booleano que indica si el grafo es dirigido (true) o no dirigido (false).
  - **`adjacencyMap`**: Un mapa de adyacencia que almacena el grafo. La clave del mapa es un vértice, y el valor es otro mapa que representa los vértices adyacentes y los pesos de las aristas correspondientes.

  - **`Network()`**: Constructor que inicializa una red (grafo) vacía.
  - **`getGraph()`**: Devuelve el mapa de adyacencia que representa el grafo.
  - **`setDirected(boolean uD_Or_D), getDirected()`**: Establece y obtiene si el grafo es dirigido.
  - **`isEmpty()`**: Comprueba si el grafo está vacío.
  - **`clear()`**: Limpia el grafo.
  - **`numberOfVertices()`**: Devuelve el número de vértices en el grafo.
  - **`numberOfEdges()`**: Devuelve el número de aristas en el grafo.
  - **`containsVertex(Vertex vertex)`**: Comprueba si el grafo contiene un vértice específico.
  - **`containsEdge(Vertex v1, Vertex v2)`**: Comprueba si el grafo contiene una arista específica.
  - **`getWeight(Vertex v1, Vertex v2)`**: Obtiene el peso de una arista específica.
  - **`setWeight(Vertex v1, Vertex v2, double w)`**: Establece el peso de una arista específica.
  - **`isAdjacent(Vertex v1, Vertex v2)`**: Comprueba si dos vértices son adyacentes.
  - **`addVertex(Vertex vertex)`**: Añade un vértice al grafo.

- **`addEdge(Vertex v1, Vertex v2, double w)`**: Añade una arista al grafo.
- **`removeVertex(Vertex vertex)`**: Elimina un vértice del grafo.
- **`removeEdge(Vertex v1, Vertex v2)`**: Elimina una arista del grafo.
- **`vertexSet()`**: Devuelve un conjunto de todos los vértices en el grafo.
- **`getNeighbors(Vertex v)`**: Devuelve un conjunto de todos los vértices adyacentes a un vértice específico.
- **`toString()`**: Devuelve una representación en cadena del grafo.
- **`iterator()`**: Devuelve un iterador sobre el conjunto de vértices del grafo.

- Clase **`MainEDALand.java`**: Esta clase sirve para representar los resultados obtenidos de los algoritmos e interactuar con el usuario a la hora de elegir que realizar.

  - **`directorioEntrada`**: Esta es una cadena que contiene la ruta del directorio donde se encuentran los archivos de entrada.
  - **`ejecutarTiempos()`**: Este método se encarga de mostrar por pantalla el tiempo que ha tardado cada algoritmo en resolverse.
  - **`mostrarPrim(ArrayList<Arista> res)`**: Muestra el resultado del algoritmo Prim.
  - **`mostrarKruskal(TreeSet<Arista> res)`**: Muestra el resultado del algoritmo Kruskal.
  - **`mostrarTSP(ArrayList<Arista> res)`**: Muestra el resultado del algoritmo TSP Greedy.
  - **`mostrarTSPOptimo(ArrayList<Arista> res)`**: Muestra el resultado del algoritmo TSP óptimo.
  - **`main(String[] args)`**: Muestra un menú que interactúa con el usuario para elegir que es lo que desea ejecutar.

- Clase **`AlgoritmosEDALand.java`**: Esta es la clase que proporcioina los métodos para realizar los algoritmos propuestos:

  - **`directorioEntrada`**: Esta es una cadena que contiene la ruta del directorio donde se encuentran los archivos de entrada.
  - **`net`**: Es una red o grafo que se representa mediante la clase Network. Esta red puede contener vértices y aristas.
  - **`pq`**: Es una cola de prioridad que se utiliza para almacenar aristas en algunos de los algoritmos.
  -
  - **`load(String nombreArchivo)`**: Este método carga un grafo desde un archivo. El archivo debe contener la información de los vértices y las aristas.
  - **`getNetwork()`**: Este método devuelve la red actual.
  - **`AlgoritmosEDALand(Network<String> red)`, `AlgoritmosEDALand(Network<String> red,`**

PriorityQueue<Arista> cola), AlgoritmosEDALand(): Son constructores de la clase que inicializan la red y la cola de prioridad.
    - **primSinPQ(String origen)**: Implementa el algoritmo de Prim para encontrar el árbol de expansión mínima sin usar una cola de prioridad.
    - **primConPQ(String origen)**: Implementa el algoritmo de Prim para encontrar el árbol de expansión mínima usando una cola de prioridad.
    - **Kruskal()**: Implementa el algoritmo de Kruskal para encontrar el árbol de expansión mínima.
    - **buscarRaiz(HashMap<String, String> verticesDesde, String vertice)**: Es un método auxiliar utilizado en el algoritmo de Kruskal para encontrar la raíz de un vértice.
    - **TSPGreedy(Network<String> netDeCiudades, String cOrigen):** Implementa una versión greedy del problema del viajante de comercio (TSP).
    - **verticeMasCercano(Network<String> netDeCiudades, HashSet<String> cVisitadas, String cActual)**: Es un método auxiliar utilizado en el algoritmo TSPGreedy para encontrar el vértice más cercano que aún no ha sido visitado.
    - **TSPSimplePaths(Network<String> netDeCiudades, String cOrigen), TSPSimplePathsAux(Network<String> netDeCiudades, String cOrigen, ArrayList<String> vertices, ArrayList<String> currentPath, ArrayList<Arista> bestPath, double bestCost):** Estos métodos implementan una versión del problema del viajante de comercio (TSP) que solo considera caminos simples.

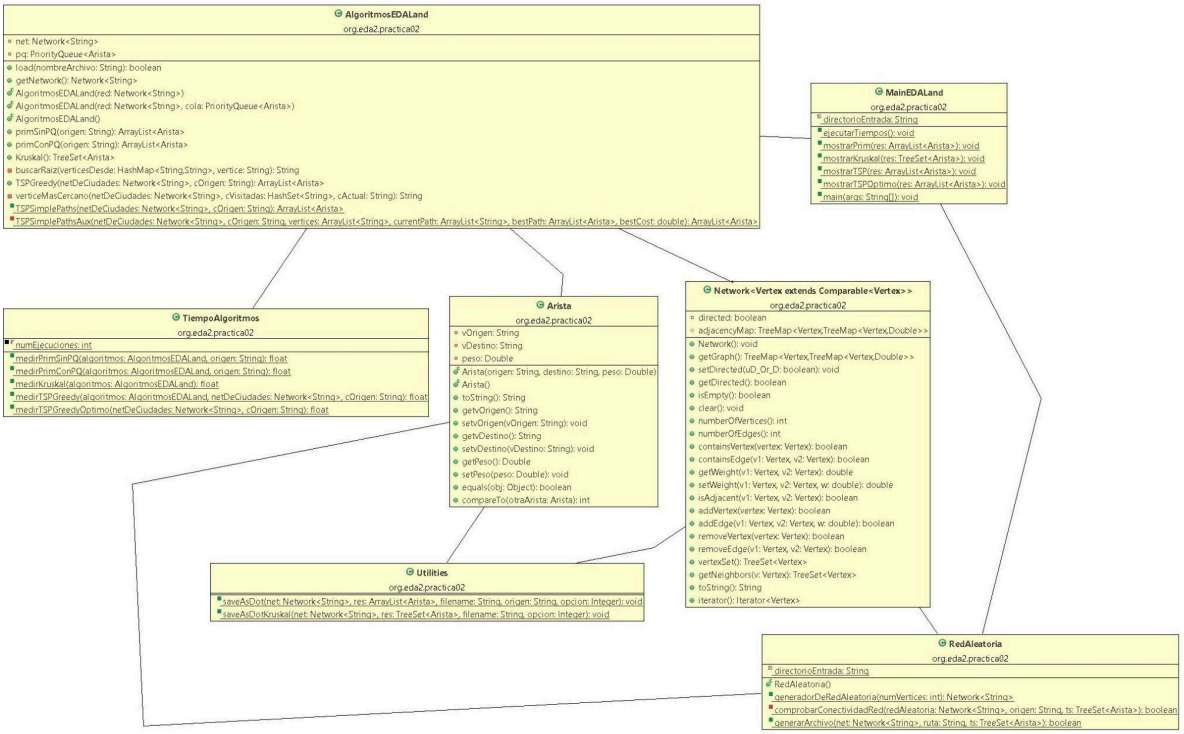
- La clase **RedAleatoria.java:** Esta es la clase que genera un grafo aleatorio y lo escribe sobre un fichero de texto
    - **directorioEntrada**: Es una cadena que representa la ruta del directorio de entrada donde se almacenarán los archivos generados.
    - **RedAleatoria()**: Es el constructor de la clase.
    - **generadorDeRedAleatoria(int numVertices)**: Este método genera una red aleatoria con un número dado de vértices. Crea aristas aleatorias entre los vértices y verifica la conectividad de la red.
    - **comprobarConectividadRed(Network<String> redAleatoria, String origen, TreeSet<Arista> ts)**: Este método verifica la conectividad de la red aleatoria. Recorre todos los vértices y verifica si cada vértice es accesible desde el vértice de origen.
    - **generarArchivo(Network<String> net, String ruta, TreeSet<Arista> ts)**: Este método genera un archivo que contiene la representación de la red. Escribe la información de los vértices y las aristas en el archivo.

- La clase **Utilities.java**: es la encargada de crear grafos en los que se destaque el camino elegido.

- **`saveAsDot(Network<String> net, ArrayList<Arista> res, String filename, String origen, Integer opcion)`**: Este método guarda un grafo en formato DOT, resaltando las aristas dadas.
- **`saveAsDotKruskal(Network<String> net, TreeSet<Arista> res, String filename, Integer opcion)`**: Este método guarda un grafo en formato DOT, resaltando las aristas dadas por el algoritmo de Kruskal.

- La clase **`AlgoritmosEDALandTest.java`** es una clase de pruebas unitarias para la clase **`AlgoritmosEDALand`**. Utiliza la biblioteca JUnit para definir y ejecutar las pruebas.
  - **`testPrimSinPQ()`**: Este método prueba el método PrimSinPQ de la clase AlgoritmosEDALand. Crea un grafo y luego llama al método PrimSinPQ con un nodo de origen específico. Luego verifica que el tamaño del resultado sea correcto y que contenga las aristas esperadas.
  - **`testPrimConPQ()`**: Este método prueba el método PrimConPQ de la clase AlgoritmosEDALand. Al igual que el método anterior, crea un grafo y luego llama al método PrimConPQ con un nodo de origen específico. Luego verifica que el tamaño del resultado sea correcto y que contenga las aristas esperadas.
  - **`testKruskal()`**: Este método prueba el método Kruskal de la clase AlgoritmosEDALand. Crea un grafo y luego llama al método Kruskal. Luego verifica que el tamaño del resultado sea correcto y que contenga las aristas esperadas.
  - **`testTSPGreedySinSolucion()`**: Este método prueba el método TSPGreedy de la clase AlgoritmosEDALand cuando no hay solución. Crea un grafo y luego llama al método TSPGreedy con un nodo de origen específico. Espera que se lance una excepción RuntimeException, lo que indica que no hay solución.
  - **`testTSPGreedyConSolucion()`**: Este método prueba el método TSPGreedy de la clase AlgoritmosEDALand cuando hay una solución. Crea un grafo y luego llama al método TSPGreedy con un nodo de origen específico. Luego verifica que el tamaño del resultado sea correcto y que contenga las aristas esperadas.

A continuación se encuentra el diagrama de clases UML asociado a esta práctica generado con AmaterasUML junto con sus asociaciones:



Fuentes bibliográficas

. Algorithm Visualizer. (s.f). Scratch paper. Recuperado de: https://algorithm-visualizer.org/scratch-paper/092cd0ec23d07f7596117ffc17f37336